

Fullstack Development

API Architectures and Design #2

Content

- What is API?
- API Architecture Styles
- **RESTful API design**
- API Security
- API Testing

RESTful API Design

RESTful API Design

Stateless

- **Single-server** environment
 - Server process any request without knowledge of previous requests
- **Multi-server** environment
 - Request not bound to a specific server
 - Able to provide service in `load-balanced` fashion
- Make APIs more `scalable` > more available
- Easy to `cache` (without state info)

RESTful API Design (2)

Most applications require some forms of `state information`



RESTful API Design (3)



RESTful API Design (4)



RESTful API Design (5)



RESTful API Design (6)

Organize API around `resources` (not actions/verbs)



Using `HTTP method` to differentiate `actions`

RESTful API Design (7)

Endpoint consistency is key, using plural form in URLs is recommended



RESTful API Design (8)

HTTP Methods and CRUD operations



RESTful API Design (9)

How to do complex query?



RESTful API Design (10)

Entities are grouped into collections



RESTful API Design (11)

Use `parameterized URIs` for identity



RESTful API Design (12)

Avoid deeply nested URLs (/collection/item/collection)



```
GET /customers/1?order=99
```

RESTful API Design (13)

Use query params for additional options or metadata



RESTful API Design (14)

Do not return `plain-text` response

- Client has to do some extra parsing/processing



RESTful API Design (15)

Response with `JSON` / `XML` / `YAML`



RESTful API Design (16)

Allow client to specify **Content-Type** header



RESTful API Design (17)

Changing API after it has been adopted is dangerous!



RESTful API Design (18)

Clients need to `update code/documents` to make app work again



RESTful API Design (19)

However, business requirements are always changed



RESTful API Design (20)

API versioning is the answer



RESTful API Design (21)

API versioning comparison



RESTful API Design (22)

API exception handling and response with proper HTTP status code



RESTful API Design (23)

HTTP status code



RESTful API Design (24)

Including links in JSON responses is a common practice

```
{
  "id": 1,
  "name": "Example",
  "links": {
    "self": "http://api.example.com/resource/1",
    "related": "http://api.example.com/resource/2"
  }
}
```

RESTful API Design (25)

Content return policies

Return the created resource

- HTTP status code: 201 Created, Location header of resource's URI
- Full detail of the newly created resource

```
HTTP/1.1 201 Created
Location: /resources/123
Content-Type: application/json
{
  "id": 123,
  "name": "New Resource"
}
```

RESTful API Design (26)

Content return policies

Do not return content

- Only a `201 Created` or `204 No Content` response, and `Location` header
- Minimizes data transfer

```
HTTP/1.1 201 Created  
Location: /resources/123
```

References

- [Best practices for REST API design](#)
- [Mastering RESTful API Design: A Practical Guide](#)
- [REST API - Best Practices - Design](#)